

**Title : Project Registration & Progress Review**

|                                                                                                                                                |                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <b>Department: Computer Engineering</b>                                                                                                        | <b>Academic Year: 2026-2027</b> |
| <b>Semester : 5</b>                                                                                                                            | <b>Group No. : SY-CS-E-12</b>   |
| <b>Project Title: Hierarchical Abstract Tree for Cross-Document Retrieval-Augmented Generation using NVIDIA CUDA-Accelerated GPU Computing</b> |                                 |
| <b>Project Area: Artificial Intelligence / Natural Language Processing (Retrieval-Augmented Generation)</b>                                    |                                 |

**Group Members Details:**

| Sr. No. | Class & Div. | Roll No. | G.R.No.  | Name of Student | Contact No. | Email ID                  |
|---------|--------------|----------|----------|-----------------|-------------|---------------------------|
| 1       | CS-E         | 29       | 12413392 | Rachit Ingole   | 8976570927  | rachit.ingole241@vit.edu  |
| 2       | CS-E         | 46       | 12414627 | Nivesh Jain     | 8208718988  | nivesh.jain24@vit.edu     |
| 3       | CS-E         | 12       | 12414068 | Hasan Rupawalla | 9156412214  | hasan.rupawalla24@vit.edu |
| 4       | CS-E         | 9        | 12414134 | Harsh Nayak     | 8999354282  | harsh.nayak24@vit.edu     |

**Name of Internal Guide: Dr. Kaushalya Thopate****Contact No. & Email ID: 9960158822  
kaushalya.thopate@vit.edu****Project approved / Not approved****Guide****Project Coordinator****Head of Department**

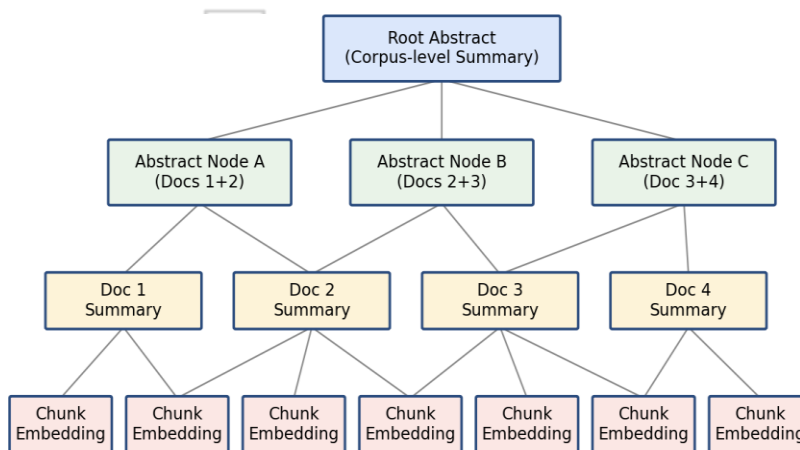
## Project Synopsis

### 1. INTRODUCTION

Large Language Models (LLMs) have transformed how users interact with information, but their static, pretrained knowledge is prone to hallucination and cannot reflect information in private or newly added documents. Retrieval-Augmented Generation (RAG) addresses this limitation by retrieving relevant passages from an external corpus at inference time and conditioning the LLM's response on that retrieved context, thereby grounding the generated answer in verifiable source material.

Conventional RAG pipelines retrieve isolated text chunks from a single document (or a flat pool of chunks from many documents) and rank them purely by chunk-level similarity to the query. This works reasonably well for narrow, token-level factual questions, but it performs poorly when a query requires reasoning across multiple documents at once — for example, comparing experimental results across several research papers, tracing how a policy clause evolved across multiple regulatory documents, or producing a high-level summary of an entire document collection.

Recent hierarchical retrieval approaches such as RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) address part of this problem by recursively clustering and summarizing chunks within a document into a tree of abstracts, allowing retrieval at multiple levels of granularity. Graph-based methods such as HippoRAG and HippoRAG2 instead build a knowledge graph of entities and relations to support multi-hop reasoning. However, tree-based methods such as RAPTOR are typically built and queried on a per-document basis and do not explicitly capture relationships between different documents, while graph-based methods can become expensive to construct and query as the size of the corpus grows, and they are less suited to producing broad, abstractive, document-level summaries.



GPU-Accelerated (NVIDIA CUDA): Embedding • Clustering • Similarity Search

fig1. Hierarchical Abstract Tree (HAT) for Cross-Document Retrieval

This project proposes a Hierarchical Abstract Tree (HAT)-based Cross-Document Retrieval-Augmented Generation system, which builds a multi-level tree of abstractive summaries that spans multiple documents, with explicit edges linking semantically related nodes drawn from different source documents. This unified structure allows the system to answer both fine-grained, token-level questions and broad, document-level or corpus-level summarization queries from the same underlying index, and to support multi-hop questions that require combining evidence from more than one document.

Because building and querying such a tree over a large document collection involves substantial embedding, clustering, and similarity-search computation, the implementation will make extensive use of NVIDIA CUDA-accelerated GPU computing. GPU acceleration is applied at every computationally intensive stage of the pipeline.

### 2. LITERATURE SURVEY

A brief survey of existing retrieval-augmented generation approaches was carried out to identify the gap that this project

addresses.

### 2.1 Naive / Flat RAG

The original RAG formulation retrieves the top-k most similar chunks to a query from a flat vector index and passes them to the LLM. It is simple and effective for single-fact lookup, but has no notion of hierarchy or cross-document structure, so it frequently misses context that is spread across multiple passages or documents.

### 2.2 RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

RAPTOR recursively clusters text chunks within a document and summarizes each cluster using an LLM, repeating the process to build a tree of abstractive summaries. Queries can then be matched against nodes at any level of the tree, allowing both fine-grained and coarse-grained retrieval. RAPTOR significantly improves performance on questions requiring broader context, but the tree is generally built per document, limiting its ability to connect related information across multiple documents.

### 2.3 HippoRAG / HippoRAG2

Inspired by models of human long-term memory, HippoRAG builds a knowledge graph of entities and relations extracted from the corpus and uses graph-based algorithms such as Personalized PageRank to retrieve relevant passages, which is particularly effective for multi-hop question answering. HippoRAG2 further improves passage integration and retrieval quality. These graph-based approaches, however, depend heavily on accurate entity/relation extraction and can face scalability challenges as the graph grows with corpus size.

### 2.4 Identified Gap

Across these approaches, there is no single method that combines (a) explicit cross-document abstractive summarization, (b) support for both token-level and document-level queries from the same index, and (c) an architecture designed from the outset for GPU-accelerated, low-latency operation at scale. This project's Hierarchical Abstract Tree (HAT) approach is designed to combine the strengths of tree-based abstraction with explicit cross-document linking, backed by NVIDIA CUDA-accelerated computation.

**Table 1. Comparison of retrieval-augmented generation approaches**

| Aspect                | RAPTOR                     | HippoRAG2                    | Proposed HAT / Psi-RAG                   |
|-----------------------|----------------------------|------------------------------|------------------------------------------|
| Scope                 | Single document tree       | Corpus-wide knowledge graph  | Cross-document abstract tree             |
| Granularity           | Chunk to document abstract | Entity / relation triples    | Chunk to corpus-level abstract           |
| Cross-document links  | Not explicit               | Implicit via shared entities | Explicit cross-document edges            |
| GPU acceleration      | Not emphasized             | Not emphasized               | NVIDIA CUDA throughout pipeline          |
| Query types supported | Mostly document-level      | Multi-hop entity QA          | Token-level + document-level + multi-hop |

## 3. PROBLEM STATEMENT

With the rapid growth of digital documents such as research papers, technical reports, product manuals, and regulatory filings, there is an increasing need to answer queries and generate summaries that require reasoning across multiple documents simultaneously. Traditional RAG pipelines retrieve isolated chunks and often miss the broader relationships between documents, leading to fragmented, incomplete, or inconsistent answers.

Existing hierarchical methods such as RAPTOR build a tree per document and do not explicitly merge information across documents, so a query that spans several documents must be handled by separately querying each document's tree and manually combining the results. Graph-based methods such as HippoRAG2 can capture some cross-document relationships through shared entities, but constructing and querying the underlying graph becomes computationally expensive as corpus size grows, and pure graph structures are less suited to producing fluent, abstractive, document-level summaries.

Furthermore, none of the existing approaches are designed from the ground up around GPU-accelerated computation, which limits their ability to scale to large, frequently updated corpora in near real time. Hence, there exists a clear need for a hierarchical retrieval architecture that (a) explicitly captures cross-document structure, (b) supports queries at multiple

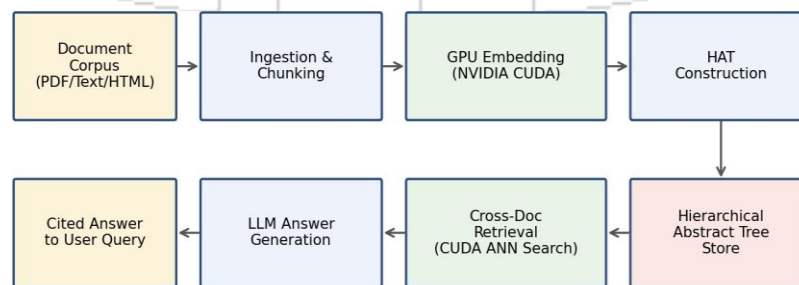
levels of granularity from a single unified index, and (c) remains efficient enough for low-latency, GPU-accelerated inference on large document collections through NVIDIA CUDA.

#### 4. GOALS / OBJECTIVES

- To design a Hierarchical Abstract Tree structure that summarizes and links content across multiple documents at increasing levels of abstraction.
- To implement cross-document retrieval that supports both token-level question answering and document-level or corpus-level summarization from the same index.
- To integrate the retrieval pipeline with a generative LLM for grounded, cited answer synthesis, reducing hallucination.
- To leverage NVIDIA CUDA and GPU acceleration for embedding generation, clustering during tree construction, and nearest-neighbor retrieval, to support large corpora with low query latency.
- To design an explicit cross-document linking mechanism that connects semantically related nodes drawn from different source documents at every level of the tree.
- To benchmark the system against existing baselines such as RAPTOR and HippoRAG2 on cross-document multi-hop QA and summarization tasks, measured using standard metrics such as F1 score.
- To provide a user interface for querying the document corpus and visualizing both the abstract tree and the specific retrieval path taken to answer a given query.
- To design the system so that it can scale incrementally as new documents are added to the corpus, without requiring the entire tree to be rebuilt from scratch.

#### 5. SYSTEM ARCHITECTURE

The proposed system is organized into four broad stages — ingestion, indexing (tree construction), retrieval, and generation — with GPU-accelerated computation embedded at every stage that involves embedding, clustering, or similarity search. fig2 below shows the overall data flow from raw document corpus to the final cited answer.



*GPU-accelerated stages (NVIDIA CUDA / cuDNN / FAISS-GPU) shown in green & pink*

*fig2. End-to-end System Architecture and Data Flow*

##### 5.1 Ingestion Layer

Responsible for parsing heterogeneous document formats (PDF, plain text, HTML), cleaning the extracted text, and splitting it into overlapping passages/chunks suitable for embedding.

##### 5.2 Indexing Layer (HAT Construction)

Responsible for generating dense embeddings for each chunk on the GPU, clustering semantically related chunks (within and across documents), and recursively summarizing clusters using an LLM to build the multi-level Hierarchical Abstract Tree, including explicit cross-document edges.

##### 5.3 Retrieval Layer

Responsible for embedding incoming queries on the GPU and performing CUDA-accelerated approximate nearest-neighbor search across tree levels to select the most relevant nodes, potentially drawn from several different documents, at

the level of abstraction appropriate to the query.

#### 5.4 Generation Layer

Responsible for assembling the retrieved multi-level, multi-document context into a prompt and invoking an LLM to produce the final answer, along with citations back to the specific source documents and passages that were used.

### 6. METHODOLOGY

1. **Document Ingestion and Chunking.** Input documents are parsed and split into overlapping passages/chunks of a fixed token budget, each of which will later be embedded and summarized. Metadata (source document ID, section, page number) is retained for later citation.
2. **GPU-Accelerated Leaf Embedding.** A sentence-embedding model executed on an NVIDIA GPU using CUDA generates dense vector representations for every chunk. Embeddings are computed in large batches to maximize GPU utilization.
3. **Hierarchical Abstract Tree Construction.** Chunks are clustered (using GPU-accelerated clustering such as CUDA-based k-means) and recursively summarized by an LLM to form parent nodes; the process repeats until a small number of root-level abstracts remain. Crucially, clustering is not restricted to a single document — chunks from different documents that are semantically close are grouped together, and cross-document edges are added between related nodes so the tree reflects relationships that span the whole corpus, not just one document.
4. **Cross-Document Retrieval.** For a given query, the system embeds the query on the GPU and traverses the tree from root to leaf using CUDA-accelerated similarity search (e.g. FAISS-GPU) to select the most relevant nodes across multiple documents at the level of abstraction appropriate to the query — root-level nodes for broad summarization queries, and leaf-level nodes for narrow factual queries.
5. **Context Assembly.** The retrieved nodes, together with their source citations, are ranked and assembled into a single prompt context, with duplicate or highly overlapping content from different documents merged to avoid redundancy.
6. **Answer Generation.** The assembled multi-level, multi-document context is passed to an LLM, which generates the final response, citing the specific source documents and passages used to produce each part of the answer.
7. **Classical / Baseline Comparison.** For evaluation, the same queries are additionally run through re-implementations of RAPTOR and HippoRAG2 on the same corpus, and the resulting answers are scored against a labeled evaluation set.
8. **Evaluation.** The system is evaluated on cross-document multi-hop QA and document-level summarization benchmarks, using average F1 score and answer-grounding accuracy as the primary metrics, compared against the RAPTOR and HippoRAG2 baselines.

### 7. MATHEMATICAL / ALGORITHMIC FORMULATION

The core retrieval step relies on measuring the semantic similarity between a query embedding and each candidate node embedding in the tree using cosine similarity:

$$\text{similarity}(q, n) = (q \cdot n) / (||q|| * ||n||)$$

where  $q$  is the query embedding and  $n$  is the embedding of a candidate tree node. During tree traversal, at each level the top- $k$  nodes by similarity score are expanded, and the search continues down the tree until the desired abstraction level (or the leaves) is reached, giving a retrieved node set  $R$  for the query.

A cross-document relevance bonus is added to encourage the retrieval of complementary evidence from different source documents, so that the final ranking score for a node  $n$  combines both semantic similarity and document diversity:

$$\text{score}(q, n) = \text{similarity}(q, n) + \text{lambda} * \text{diversity\_bonus}(n, R)$$

where  $\text{diversity\_bonus}$  rewards nodes originating from documents not yet represented in the currently retrieved set  $R$ , and  $\text{lambda}$  is a tunable weighting hyperparameter. Finally, the LLM's generation is conditioned on the assembled context  $C$  derived from  $R$ , and the model is prompted to produce an answer  $y$  together with citations to the originating documents:

$$y = \text{LLM\_generate}(\text{query} = q, \text{context} = C(R))$$

All of the above similarity computations, the clustering used to build the tree, and the batched embedding calls are executed on NVIDIA GPUs using CUDA to keep both index construction and query-time latency low.

## 8. TECHNICAL DETAILS

1. **Programming Language & Platform:** Python, using PyTorch for model inference and training-free experimentation.
2. **GPU Acceleration (NVIDIA CUDA):** the NVIDIA CUDA toolkit and cuDNN will be used to accelerate embedding generation, clustering, and similarity search; FAISS-GPU or RAPIDS cuML will be used for GPU-based nearest-neighbor search and clustering, and mixed-precision (FP16) inference will run on CUDA-enabled GPUs to maximize throughput.
3. **Embedding Model:** a pretrained sentence-embedding / transformer model executed on GPU to embed both chunks and incoming queries into a shared vector space.
4. **LLM for Summarization & Generation:** an open-source or API-based LLM used to generate abstractive summaries at each tree level and the final grounded answer with citations.
5. **Tree Storage:** the Hierarchical Abstract Tree is stored using an in-memory tree/graph structure with explicit cross-document edges, persisted to disk for reuse across sessions.
6. **Vector Index:** a GPU-resident vector index (FAISS-GPU) stores node embeddings at every tree level for fast approximate nearest-neighbor search.
7. **Input Parameters:** document corpus (PDF/text/HTML), query text, and desired level of abstraction (token-level, document-level, or corpus-level).
8. **Retrieval Method:** multi-level similarity search combined with cross-document link traversal and a diversity-aware ranking score.
9. **Visualization:** tools to display the abstract-tree structure and highlight the retrieval path taken for a given query, built using a graph-visualization library.
10. **User Interface (UI):** a web-based interface (React frontend with a FastAPI backend) for corpus upload, querying, and tree visualization.
11. **Performance & Optimization:** batched GPU inference, FP16 mixed-precision computation on NVIDIA CUDA hardware, embedding caching, and incremental tree updates to reduce query latency and re-indexing cost.
12. **Deployment:** the system will be containerized (Docker) with GPU passthrough for NVIDIA CUDA, allowing deployment on a local workstation GPU or a cloud GPU instance.
13. **Testing & Validation:** the system will be validated on cross-document multi-hop QA benchmarks, comparing average F1 scores and retrieval latency against RAPTOR and HippoRAG2.

## 9. DATASET AND EVALUATION METRICS

The system will be evaluated on multi-document question-answering and summarization datasets that require combining information from more than one source document, supplemented by a small custom corpus assembled from openly available research papers and technical reports for qualitative testing.

### 9.1 Evaluation Metrics

- Average F1 score on cross-document multi-hop QA benchmarks, comparing predicted answers to reference answers.
- Retrieval precision/recall — whether the retrieved nodes actually contain the evidence needed to answer the query.
- Answer-grounding accuracy — whether every claim in the generated answer can be traced to a cited source passage.
- Query latency (milliseconds) and index build time, measured with and without NVIDIA CUDA GPU acceleration, to quantify the speed-up obtained from GPU computation.
- ROUGE-based summary quality scores for document-level and corpus-level summarization queries.

## 10. CHALLENGES AND SOLUTION

1. Constructing accurate cross-document links requires reliable semantic similarity computation across large corpora, which is computationally expensive.
2. To address this, NVIDIA CUDA-accelerated batch embedding and approximate nearest-neighbor search will be

used to keep tree construction efficient even as corpus size grows.

3. Balancing tree abstraction levels is difficult — too coarse loses detail, while too fine reduces retrieval efficiency and increases the number of LLM summarization calls — and will require careful tuning of tree depth and cluster size.

4. Repeated LLM summarization at each tree level increases inference cost; this will be mitigated through GPU batching and caching of intermediate summaries so unchanged branches of the tree are not regenerated.

5. Ensuring retrieval remains efficient as the corpus grows will be handled through incremental tree updates — inserting or re-clustering only the affected branches — rather than full tree rebuilds.

6. Avoiding duplicate or contradictory information from different documents in the final answer will be addressed through the diversity-aware ranking score and a de-duplication step during context assembly.

7. Limited availability of GPU memory when processing very large corpora will be managed through streaming/batched processing and, where necessary, multi-GPU distribution of the embedding and clustering workload.

## 11. USE CASES

### 1. Research Literature Review

Helps researchers synthesize findings across many papers, answering both specific factual queries (e.g. "what dataset did paper X use") and broad summarization requests (e.g. "summarize the main approaches to this problem across the last five years") with citations to the specific source papers.

### 2. Enterprise Knowledge Base QA

Allows organizations to query across large sets of internal documents (reports, manuals, policies) with grounded, cited answers, reducing time spent manually searching across multiple files.

### 3. Legal / Compliance Document Analysis

Supports cross-referencing clauses and facts across multiple contracts or regulatory documents, helping identify inconsistencies or overlapping obligations.

### 4. Educational Content Summarization

Assists students in synthesizing information drawn from multiple textbooks or papers on a given topic, producing a coherent overview with references back to each source.

### 5. High-Throughput Corpus Analytics

Enables analysts to quickly generate ranked, cross-document summaries over large document sets (e.g. news archives, patent filings), leveraging GPU acceleration to keep processing time practical.

## 12. EXPECTED OUTCOMES

- A working prototype of a Hierarchical Abstract Tree-based cross-document retrieval-augmented generation system.
- A GPU-accelerated (NVIDIA CUDA) embedding, clustering, and retrieval pipeline demonstrating measurable latency improvement over CPU-only execution.
- Quantitative benchmark results comparing the proposed system against RAPTOR and HippoRAG2 on cross-document multi-hop QA and summarization tasks.
- A visualization interface that displays the abstract tree and the retrieval path used to answer a given query.
- A documented, reusable pipeline that can be pointed at new document corpora for future extension.

## 13. PROJECT TIMELINE

The project is planned to be executed over the semester in six phases, as summarized below.

| Phase   | Duration  | Deliverable                                                                           |
|---------|-----------|---------------------------------------------------------------------------------------|
| Phase 1 | Weeks 1-3 | Literature survey, dataset collection, corpus preprocessing pipeline                  |
| Phase 2 | Weeks 4-6 | GPU-accelerated embedding pipeline and chunk-level indexing (NVIDIA CUDA + FAISS-GPU) |

|         |             |                                                                     |
|---------|-------------|---------------------------------------------------------------------|
| Phase 3 | Weeks 7-9   | Hierarchical Abstract Tree construction with cross-document linking |
| Phase 4 | Weeks 10-12 | Cross-document retrieval module and LLM-based answer generation     |
| Phase 5 | Weeks 13-14 | Integration with UI, visualization of tree and retrieval path       |
| Phase 6 | Weeks 15-16 | Benchmarking against RAPTOR / HippoRAG2, report and documentation   |

#### 14. FUTURE SCOPE

In future development, the system can be extended to support multi-modal documents (tables, images, charts), further GPU optimization using NVIDIA TensorRT for lower-latency inference, streaming/incremental corpus updates, and integration with enterprise search systems. Multi-GPU and distributed CUDA-based processing can also be explored to scale the Hierarchical Abstract Tree to enterprise-sized document collections. Additional directions include personalized retrieval based on user history, active-learning-based tree refinement, and support for multilingual document corpora.

- Integration of Density Functional style scientific-document parsing for domain-specific corpora (e.g. scientific PDFs with equations and figures).
- Adaptive tree depth that automatically adjusts based on corpus size and query pattern history.
- Support for continuous ingestion pipelines that update the Hierarchical Abstract Tree as new documents arrive, using GPU-accelerated incremental clustering.
- Fine-tuning the embedding model on domain-specific corpora to further improve cross-document link quality.

#### 15. CONCLUSION

The proposed project presents a Hierarchical Abstract Tree-based Cross-Document Retrieval-Augmented Generation system that addresses the limitations of existing single-document and graph-based RAG approaches. By explicitly linking semantically related content across documents at multiple levels of abstraction, and by leveraging NVIDIA CUDA-accelerated GPU computing for embedding, clustering, and retrieval, the system aims to efficiently support both fine-grained and broad multi-document queries, providing accurate, cited answers across large document corpora while remaining scalable, extensible, and practical for real-world use.

#### 16. TEAM ROLES AND RESPONSIBILITIES

The project work has been divided among the four group members based on the major modules of the system, with regular integration checkpoints so that all modules interoperate correctly. Responsibilities may be adjusted during the semester as the project progresses and specific modules require additional effort.

| Member          | Roll No. | Primary Responsibility                                                   |
|-----------------|----------|--------------------------------------------------------------------------|
| Rachit Ingole   | 29       | Ingestion pipeline, document parsing/chunking, and metadata management   |
| Nivesh Jain     | 46       | GPU-accelerated embedding and clustering (NVIDIA CUDA, FAISS-GPU)        |
| Hasan Rupawalla | 12       | Hierarchical Abstract Tree construction and cross-document linking logic |
| Harsh Nayak     | 9        | Retrieval module, LLM-based generation, evaluation, and UI integration   |

#### 17. ASSUMPTIONS AND CONSTRAINTS

##### 17.1 Assumptions

- A CUDA-capable NVIDIA GPU will be available (locally or via a cloud instance) for development, embedding generation, and evaluation.
- The document corpus used for development and evaluation will primarily be text-extractable (i.e. not scanned

images requiring OCR).

- Access to a pretrained embedding model and an LLM (open-source or API-based) will be available for summarization and generation.
- The evaluation benchmarks used for comparison against RAPTOR and HippoRAG2 are representative of typical cross-document, multi-hop query patterns.

### 17.2 Constraints

- GPU memory capacity will limit the maximum batch size used during embedding and clustering, particularly for very large corpora.
- The quality of abstractive summaries at each tree level is bounded by the capability of the underlying LLM used for summarization.
- Project development is constrained to the duration of Semester 4 of the academic year 2025-2026, limiting the scope of experimentation and the number of baseline systems that can be re-implemented and compared.
- Real-time collaborative editing of the corpus (multiple users modifying documents simultaneously) is outside the scope of the current implementation.

## 18. RISK ANALYSIS

1. **Risk:** GPU resource unavailability or insufficient VRAM during peak experimentation. Mitigation: use gradient/embedding batching, smaller embedding models, and cloud GPU fallback.
2. **Risk:** LLM-generated summaries at intermediate tree levels may drift from the source content (summarization hallucination). Mitigation: periodic validation of intermediate summaries against source chunks and citation-consistency checks.
3. **Risk:** Benchmark datasets may not fully represent real-world cross-document queries. Mitigation: supplement standard benchmarks with a small hand-curated evaluation set built from the team's own multi-document corpus.
4. **Risk:** Integration delays between the retrieval, generation, and UI modules. Mitigation: define clear API contracts between modules early and integrate incrementally at the end of each project phase.

## 19. NOVELTY OF THE PROPOSED WORK

The novelty of this project lies in combining three elements that existing systems address only partially: explicit cross-document abstraction, unified multi-granularity querying, and GPU-first system design.

- Unlike RAPTOR, which builds an abstract tree per document, the proposed Hierarchical Abstract Tree explicitly clusters and links content across documents at every level, so a single query can retrieve complementary evidence from multiple sources without manually querying each document separately.
- Unlike graph-based methods such as HippoRAG2, which primarily target multi-hop entity reasoning, the tree structure natively supports both narrow, token-level lookup and broad, abstractive, corpus-level summarization from the same index.
- Unlike most existing RAG research prototypes, the system is designed from the outset around NVIDIA CUDA-accelerated GPU computation at every stage — embedding, clustering, and retrieval — rather than treating GPU usage as an afterthought, making the design more directly applicable to production-scale deployments.

## 20. RESOURCE / COST ESTIMATION

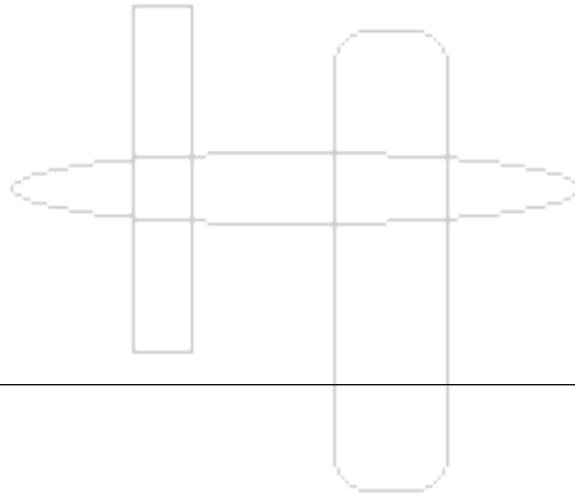
The table below summarizes the resources anticipated for developing and evaluating the prototype. As the project relies primarily on open-source software and freely available research datasets, the main cost driver is GPU compute time for embedding and LLM inference.

| Resource                      | Type                     | Notes                                                           |
|-------------------------------|--------------------------|-----------------------------------------------------------------|
| NVIDIA CUDA-capable GPU       | Hardware (owned / cloud) | Used for embedding, clustering, and ANN search acceleration     |
| Cloud GPU instance (optional) | Cloud compute            | Fallback for large-scale benchmarking beyond local GPU capacity |

|                            |                  |                                                                      |
|----------------------------|------------------|----------------------------------------------------------------------|
| LLM API / hosted inference | Software service | Used for abstractive summarization and final answer generation       |
| Open-source libraries      | Software         | PyTorch, FAISS-GPU / RAPIDS cuML, React, FastAPI — no licensing cost |

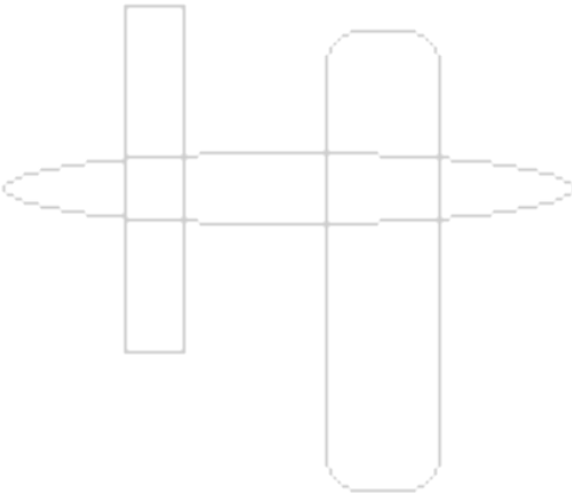
## 21. REFERENCES

- Sarthi, P. et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," 2024.
- Gutierrez, B. et al., "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models," 2024.
- Gutierrez, B. et al., "HippoRAG2: Deeper Passage Integration for Long-Term Memory," 2025.
- Lewis, P. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020.
- Johnson, J., Douze, M., Jegou, H., "Billion-scale similarity search with GPUs (FAISS)," 2019.
- NVIDIA Corporation, "CUDA Toolkit Documentation."



|           |                   |                                  |                    |
|-----------|-------------------|----------------------------------|--------------------|
| Group No. |                   |                                  |                    |
| Activity  | Review Schedule   | Progress Review Report submitted | Signature of Guide |
| Review 1  | Mid Sem. Semester | Yes / No                         |                    |
| Review 2  | End of Semester   | Yes / No                         |                    |

Format of Progress Review Report:

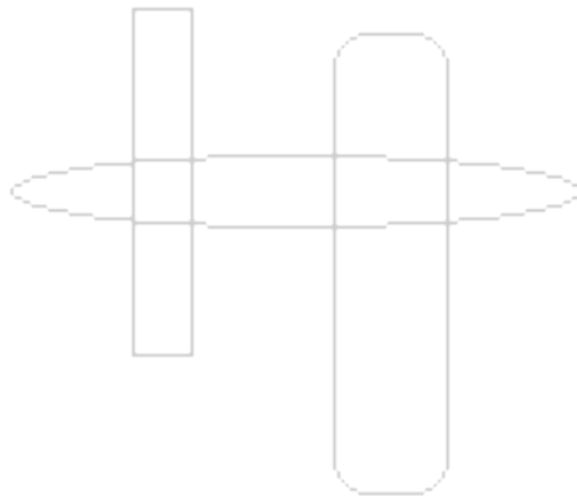
|                                                                                      |                   |              |
|--------------------------------------------------------------------------------------|-------------------|--------------|
| <b>Review No.: 1</b>                                                                 | <b>Group No.:</b> | <b>Date:</b> |
| <b>Progress Review Report</b>                                                        |                   |              |
|  |                   |              |
| <b>Signature of Guide:</b>                                                           |                   |              |

**Review No.: 2**

**Group No.:**

**Date:**

**Progress Review Report**



**Signature of Guide:**